



Python Programming Fundamentals

Searching Algorithms

Programming Fundamentals Team

SETU

2026-06-12



Outline

1. Two Main Search Algorithms
2. Linear Search
3. Binary Search
4. Binary Search Trace
5. Python's `bisect` Module
6. Searching Sorted Objects
7. Complexity Summary



Outline

1. Two Main Search Algorithms

2. Linear Search

3. Binary Search

4. Binary Search Trace

5. Python's bisect Module

6. Searching Sorted Objects

7. Complexity Summary



1. Two Main Search Algorithms

Linear Search	Binary Search
Works on any list	Requires sorted list
$O(n)$ — checks each item	$O(\log n)$ — halves search space
Simple to implement	More complex
Best for small/unsorted data	Best for large sorted data



Outline

1. Two Main Search Algorithms

2. Linear Search

3. Binary Search

4. Binary Search Trace

5. Python's `bisect` Module

6. Searching Sorted Objects

7. Complexity Summary



2. Linear Search

```
def linear_search(lst: list, target) -> int:
    """Return index of target, or -1 if not found."""
    for i, item in enumerate(lst):
        if item == target:
            return i
    return -1
```

```
numbers = [15, 8, 3, 42, 7, 19]
idx = linear_search(numbers, 42)
print(idx)    # 3
```

Searching objects

```
def find_product_by_name(products: list, name: str):
    for p in products:
        if p.get_name().lower() == name.lower():
```



2. Linear Search

```
        return p  
    return None
```



Outline

1. Two Main Search Algorithms
2. Linear Search
- 3. Binary Search**
4. Binary Search Trace
5. Python's bisect Module
6. Searching Sorted Objects
7. Complexity Summary



3.1 Divide and conquer on a sorted list

```
def binary_search(lst: list, target) -> int:
    """Return index of target in sorted lst, or -1."""
    low, high = 0, len(lst) - 1
    while low <= high:
        mid = (low + high) // 2
        if lst[mid] == target:
            return mid
        elif lst[mid] < target:
            low = mid + 1 # target is in right half
        else:
            high = mid - 1 # target is in left half
    return -1
```

```
sorted_nums = [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]
print(binary_search(sorted_nums, 23)) # 5
print(binary_search(sorted_nums, 10)) # -1
```



Outline

1. Two Main Search Algorithms
2. Linear Search
3. Binary Search
- 4. Binary Search Trace**
5. Python's `bisect` Module
6. Searching Sorted Objects
7. Complexity Summary



4. Binary Search Trace

Searching for 23 in [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]

Step 1: low=0, high=9, mid=4 → lst[4]=16 < 23 → low=5

Step 2: low=5, high=9, mid=7 → lst[7]=56 > 23 → high=6

Step 3: low=5, high=6, mid=5 → lst[5]=23 == 23 → return 5

Only 3 steps to search 10 items! Linear would take up to 10.

For 1,000,000 items: binary search takes at most $\log_2(1,000,000) \approx 20$ steps.



Outline

1. Two Main Search Algorithms
2. Linear Search
3. Binary Search
4. Binary Search Trace
- 5. Python's `bisect` Module**
6. Searching Sorted Objects
7. Complexity Summary



5. Python's bisect Module

```
import bisect

sorted_prices = [1.20, 1.99, 2.50, 3.00, 4.50]

# Find insertion point for 2.00
idx = bisect.bisect_left(sorted_prices, 2.00)
print(idx)    # 2 ← insert at position 2

# Insert while keeping sorted
bisect.insort(sorted_prices, 2.00)
print(sorted_prices)    # [1.20, 1.99, 2.00, 2.50, 3.00, 4.50]

bisect uses binary search internally —  $O(\log n)$  lookup.
```



Outline

1. Two Main Search Algorithms
2. Linear Search
3. Binary Search
4. Binary Search Trace
5. Python's bisect Module
- 6. Searching Sorted Objects**
7. Complexity Summary



6. Searching Sorted Objects

```
# Sort by price first
products = sorted(shop.get_all_products(),
                  key=lambda p: p.get_price())

prices = [p.get_price() for p in products]

def find_by_price(products: list, target_price: float):
    prices = [p.get_price() for p in products]
    idx = binary_search(prices, target_price)
    if idx != -1:
        return products[idx]
    return None

# Or use bisect
import bisect
idx = bisect.bisect_left(prices, 2.50)
```



6. Searching Sorted Objects

```
if idx < len(prices) and prices[idx] == 2.50:  
    print(products[idx])
```



Outline

1. Two Main Search Algorithms
2. Linear Search
3. Binary Search
4. Binary Search Trace
5. Python's bisect Module
6. Searching Sorted Objects
- 7. Complexity Summary**



7. Complexity Summary

n items	Linear	Binary
10	10 steps max	4 steps max
1,000	1,000 steps	10 steps
1,000,000	1,000,000 steps	20 steps
1,000,000,000	1 billion steps	30 steps

Binary search is **dramatically** faster on large sorted data.



Thanks for Watching - Any questions?

